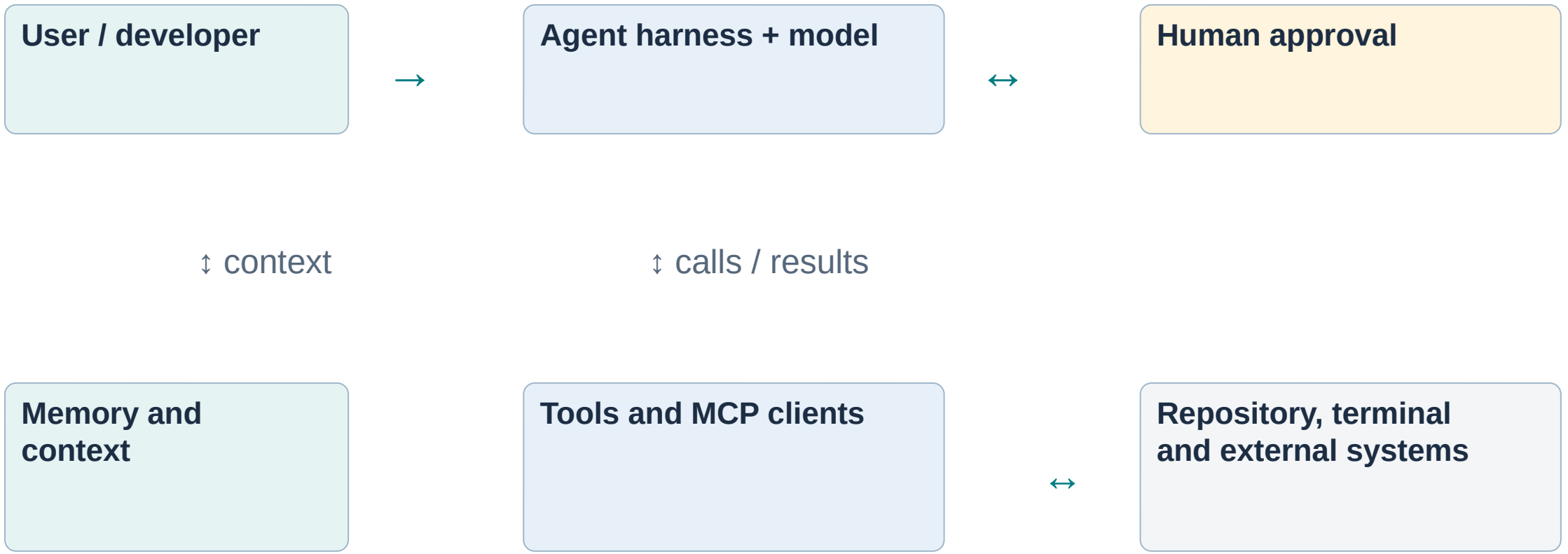
Coding Agents

Architecture, orchestration, tools, memory and benchmarks

COT4930/COT5930 Security Engineering with GenAI
Instructor: Dr. Ionut Cardei

High-level architecture



The harness manages state, permissions, tool execution and stopping conditions.

The agent loop

1. Interpret the request, constraints and repository instructions.

2. Inspect files and gather observations through tools.

3. Choose an action, execute it and incorporate the result.

4. Validate the change, revise on failure and report evidence.

The controller enforces budgets, permissions and termination.

Conceptual loop; see ReAct: https://arxiv.org/abs/2210.03629

Tools: grounding actions in the environment

- Read and search: directory listing, file reads, rg, symbol or language-server search.
- Edit: structured patches or targeted file edits. Verify the diff after applying changes.
- Execute: shell commands, build systems, test runners and debuggers.
- Tools return observations and errors. The model must interpret them and decide what to do next.
- The available tool interface and sandbox vary by product.

MCP: a standard interface to external capabilities

- A host creates MCP clients that connect to one or more MCP servers.
- Servers can expose tools, resources and prompts through a standard protocol.
- The agent invokes a server tool through its client. The server executes it and returns a result.
- MCP defines an interface, not an agent planner or a security guarantee.
- Treat server outputs as untrusted data and enforce authorization at execution.

Source: https://modelcontextprotocol.io/specification/

Skills and reusable procedures

- A skill packages task-specific instructions, and may include scripts or references.
- The agent selects relevant instructions and loads details when needed.
- Examples: a repository release procedure, a document workflow or a domain-specific review checklist.
- Skills guide behavior. A tool or script performs an operation; an MCP server can expose that tool.
- Instructions can become stale or conflict with higher-priority user and system constraints.

Memory and state

- Working context: current instructions, dialogue, recent actions and observations in the model input.
- External task state: plan, test results, repository state, diffs and execution logs.
- Episodic memory: records of prior tasks or interactions, when provided by the product.
- Semantic or procedural memory: persistent facts, preferences and reusable instructions.
- Model weights encode learned patterns, but are not a reliable, current project database.

Context compression: benefits and failure modes

- The context window is finite; long sessions require selection, retrieval or summarization.
- Compression can omit constraints, decisions, exact error messages or failed approaches.
- A summary may introduce errors, lose provenance or promote an unverified claim to fact.
- Mitigate with structured state: goals, invariants, changed files, tests and unresolved questions.
- Re-read authoritative files and tool outputs before consequential actions.

Orchestration algorithms

- ReAct: alternate reasoning and actions; adapt after each observation.
- Plan-and-execute: construct a plan, execute steps, then revise as evidence changes.
- Graph or state machine: explicit states, transitions, retries and approval gates.
- Search over candidates: generate, test and select patches; more robust but more expensive.
- Multi-agent delegation: split work, isolate edits and integrate results; coordination adds overhead.

ReAct: https://arxiv.org/abs/2210.03629 | SWE-agent: https://arxiv.org/abs/2405.15793

A concrete repository workflow

- Inspect: git status; rg for relevant symbols; read project instructions and existing tests.
- Change: use a patch tool for a focused edit, preserving unrelated user changes.
- Verify: run the relevant tests and build commands, then inspect git diff.
- Git: inspect history, status and diff; create a branch or commit when authorized.
- Human review: surface uncertainty and obtain approval at configured decision points.

Human-in-the-loop control

- Clarification: ask when a missing requirement blocks a consequential design choice.
- Approval: pause before actions outside the granted scope, such as publishing or changing external systems.
- Review: show the patch, test outcomes and known limitations.
- Recovery: return control after repeated failure, unexpected state or an ambiguous result.
- A human review step improves accountability but does not prove correctness.

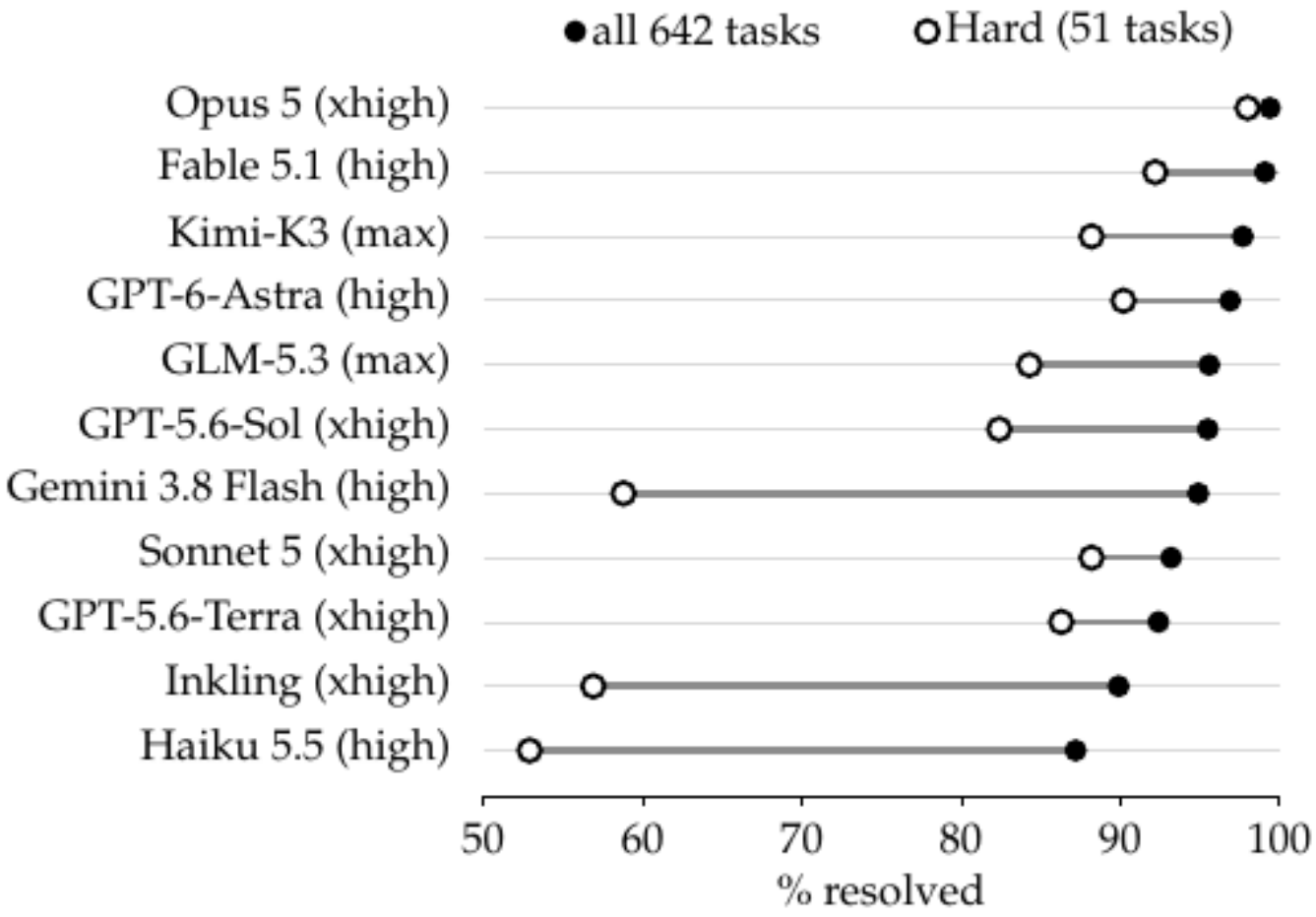
Risks of coding agents

- Incorrect or incomplete changes can pass weak or narrowly scoped tests.
- Prompt injection in repository files, logs or web content can redirect the agent.
- Secrets and sensitive code may be exposed to tools, services or external logs.
- Commands can damage files, alter environments or make unauthorized external changes.
- Dependency changes may introduce vulnerabilities, license issues or unreviewed behavior.

Best practices

- Define the task, acceptance criteria and authorization boundaries.
- Use a sandbox and least-privilege credentials; restrict network and sensitive data.
- Review untrusted instructions and tool outputs as data. Keep secrets out of prompts and logs.
- Inspect the diff; run relevant tests, static checks and security checks.
- Use human review for significant changes and monitor the deployed result.

SWE-Bench Pro V2: October 2026



Coding agent benchmarks for solving complex, real-world software issues in repositories, using tests to check whether their code changes work against verified solutions.

Benchmark link

Dan et al., 2026, Fig. 1: https://arxiv.org/abs/2610.24206

Leakage controls can change scores



731 tasks in Pro Verified.

Light bars: baseline; dark bars: verified setting.

Scores reflect both anti-hacking controls and task refinements.

The chart uses a truncated vertical axis.

Zheng et al., 2026, Fig. 1: https://arxiv.org/abs/2609.08149

Refactoring results depend on the harness

	Overall	Avg. Steps	Avg. Cost	Per-Language Resolve Rate						
				Py	Java	TS	Go	C	C++	Rust
Mini-SWE-Agent										
Proprietary										
Gemini-3-Pro	26.5	58.0	$0.60	17.2	15.4	14.3	26.1	50.0	45.5	27.3
Claude Sonnet 4.6	30.6	99.5	$2.32	13.8	19.2	32.1	13.0	65.0	40.9	40.9
GPT-5.2	21.8	25.2	$0.19	17.2	15.4	21.4	13.0	45.0	31.8	13.6
Open-weight										
GLM-5	22.9	108.9	$0.10	13.8	7.7	25.0	21.7	50.0	22.7	27.3
Kimi-K2.5	26.5	85.3	$0.37	17.2	23.1	21.4	17.4	60.0	31.8	22.7
Qwen3.5	20.6	155.4	$0.93	17.2	7.7	10.7	13.0	45.0	27.3	31.8
OpenHands										
Proprietary										
Gemini-3-Pro	19.4	51.2	$1.49	13.8	19.2	0.0	8.7	45.0	36.4	22.7
Claude Sonnet 4.6	38.8	117.9	$4.77	17.2	30.8	53.6	26.1	50.0	36.4	63.6
GPT-5.2	41.2	115.1	$3.60	48.3	19.2	35.7	26.1	75.0	36.4	54.5
Open-weight										
GLM-5	36.5	114.2	$0.24	20.7	34.6	28.6	34.8	65.0	45.5	36.4
Kimi-K2.5	32.9	99.6	$0.72	24.1	30.8	10.7	43.5	70.0	45.5	18.2
Qwen3.5	36.5	141.2	$0.78	37.9	26.9	17.9	39.1	65.0	54.5	22.7

Table 3 Resolve rates (%) of all evaluated models across scaffolds on SWE-Bench ProMax.

170 multilingual refactoring tasks.

GPT-5.2: 21.8% with mini-SWE-agent; 41.2% with OpenHands.

Results reflect the paper's evaluated configurations.

Shi et al., 2026, Table 3: https://arxiv.org/abs/2608.09802

How to interpret benchmark scores

- A score measures a model, harness, tools, budget and test suite together.
- Pass@1 reports the proportion solved in one attempt under the stated evaluation protocol.
- Benchmark datasets and task difficulty differ; percentages are not directly comparable across tests.
- Tests may miss defects. Leakage, stale tasks, selection effects and cost limits affect validity.
- Use benchmarks alongside evaluation on your own repositories and review requirements.

Selected sources

Dan et al. SWE-Bench Pro V2 (2026)
https://arxiv.org/abs/2610.24206

Zheng et al. SWE-Bench Pro Verified (2026)
https://arxiv.org/abs/2609.08149

Shi et al. SWE-Bench ProMax (2026)
https://arxiv.org/abs/2608.09802

Yao et al. ReAct (2022) https://arxiv.org/abs/2210.03629

Yang et al. SWE-agent (2024) https://arxiv.org/abs/2405.15793

MCP specification https://modelcontextprotocol.io/specification/